AEM341 Project Part1

Collin Atkins

Department of Aerospace Engineering and Mechanics at the University of Alabama, Tuscaloosa, AL, 35487, USA

I. Objective

The objective of this project was to develop a MATLAB program for bidirectional bending calculations of an unsymmetrical L-shaped cantilever beam using Bernoulli-Euler beam theory. The MATLAB program was required to determine the following for a general L-shaped cross-section: centroid, area moments of inertia, product of inertia, k-values, and neutral-axis angle. This general program was then applied to the specific cantilever beam in Fig. 2 to calculate the bending stresses at the ten strain-gage locations in Table 2, the horizontal and vertical displacements at the five required cross-sections, and the required plots using a discretization of 1 mm along the beam.

II. Theory and Method

Bending in the beam occurs with coupling between the y- and z-directions. This is because the beam cross-section is unsymmetrical. The L-shaped beam section was split into two rectangles so that both the total area and centroid could be found by the standard composite-area method. The centroidal area moments of inertia I_y and I_z were determined using the parallel-axis theorem after locating the centroid. The product of inertia I_{yz} was determined using the parallel-axis theorem as well.

The bending stress at selected points on the cross-section was then computed using the bidirectional bending formulation. Since the beam is a cantilever with a tip load, the internal bending moment varies linearly from a maximum magnitude at the fixed end to a magnitude of zero at the free end of the beam. The MATLAB program also used Bernoulli-Euler beam theory to calculate horizontal and vertical beam deflections along the beam's span. Bending-moment state and centroidal section properties were then used to determine the neutral-axis angle. This approach creates a general program for an L-shaped section and then applies it to a specific beam and loading case.

III. MATLAB Program

The MATLAB program was organized into modular functions so that the following could be tailored to any specific cantilever beam without rewriting the main general analysis of the program: geometry, material properties, beam length, and tip loads. The main script defined the input data and then calculated the cross-section properties, bending moments, stresses, and deflections. The inputs were validated, section properties were computed, stress at selected cross-section points was evaluated, and displacements along the beam were computed using separate helper functions.

For the specific case in Task 2, the MATLAB program used the dimensions shown in Fig. 2, a beam length of 1 m, an aluminum elastic modulus of 68.9 GPa, and a 12-lb tip load. The beam was discretized at 1 mm intervals so that the required stress and displacement plots could be generated from the program with respect to the horizontal beam location. Also, the ten strain-gage locations in Table 2 were evaluated, and the five required cross-section displacement locations were reported by the program.

IV. Input Data

The specific cantilever beam analyzed used the Fig. 2 cross-section dimensions of h_1=0.03 m, h_2=0.06 m, t_1=0.002 m, and t_2=0.002 m. The beam length was defined as 1 m, the material property was defined as aluminum with E=68.9 GPa, and the applied tip load was defined as 12 lb. The beam and material input data are listed in Table 1. The required stress evaluation points were the ten strain-gage locations listed in Table 2. The displacement results were recorded at the first five strain-gage x-locations.

Table 1. Beam and Material Input Data

Parameter	Value
Vertical leg height, h_1	0.030 m
Horizontal leg length, h_2	0.060 m
Vertical leg thickness, t_1	0.002 m
Horizontal leg thickness, t_2	0.002 m
Beam length, L	1.000 m
Elastic modulus, E	68.9 GPa
Tip load	12 lb
Load direction	-z direction
Span discretization	1 mm

Table 2. Strain-Gage Locations

Strain Gage	x Value (m)	y Value (m)	z Value (m)
1	0.0348	0.0000	0.0150
2	0.2761	0.0000	0.0150
3	0.5047	0.0000	0.0150
4	0.7333	0.0000	0.0150
5	0.9746	0.0000	0.0150
6	0.0348	0.0200	0.0000
7	0.2761	0.0200	0.0000
8	0.5047	0.0200	0.0000
9	0.7333	0.0200	0.0000
10	0.9746	0.0200	0.0000

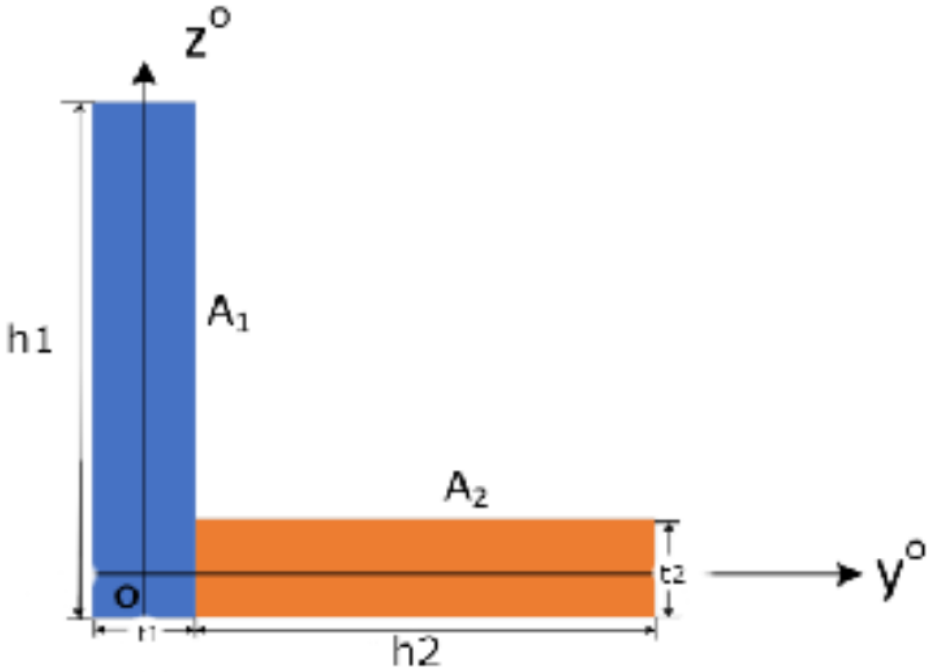


Fig. 1. General L-shaped cross-section geometry used for bidirectional bending analysis

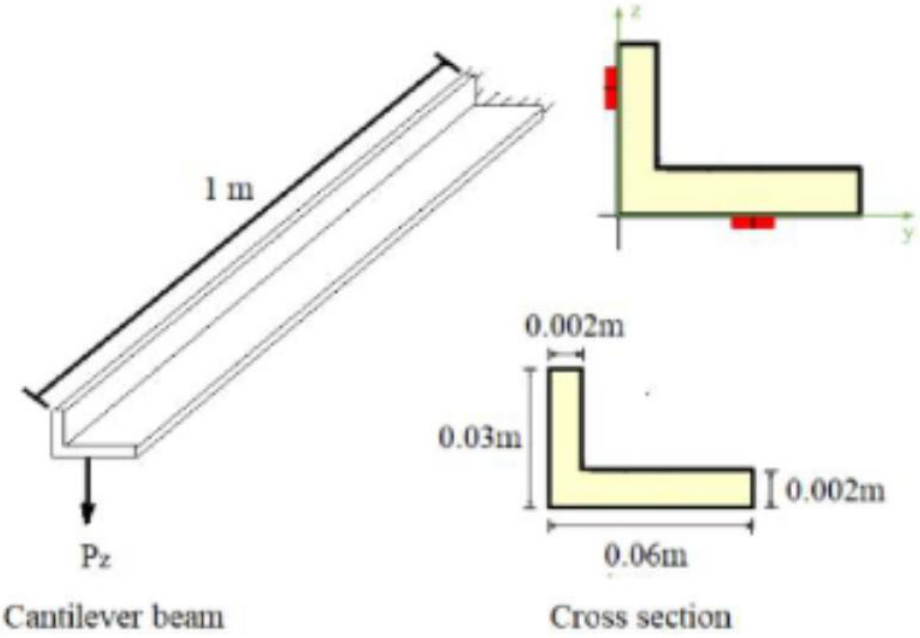


Fig. 2. Specific Cantilever Beam Geometry and Loading for Task 2

V. Results

The MATLAB program successfully computed the cross-section properties of the L-shaped section. The following were calculated by the program and are listed in Table 3: area, centroid location, area moments of inertia, product of inertia, k-values, and neutral-axis angle. The centroid was located at $\bar{y}=2.166667 \times 10^{-2}$ m and $\bar{z}=5.666667 \times 10^{-3}$ m. The corresponding

centroidal properties were also found to be I_y=1.238000 x 10^{-8} m^4, I_z=7.446000 x 10^{-8} m^4, I_{yz}=-1.736000 x 10^{-8} m^4. The neutral-axis angle was calculated to be 166.876195°. The L-shaped cross-section with the calculated centroid and the two selected stress-evaluation points is shown in Fig. 3.

Table 3. Calculated Cross-Section Properties for the L-Shaped Section

Property	Value
Area	1.800000 x 10^{-4} m^2
Centroid, $\bar{y}$	2.166667 x 10^{-2} m
Centroid, $\bar{z}$	5.666667 x 10^{-3} m
I_y	1.238000 x 10^{-8} m^4
I_z	7.446000 x 10^{-8} m^4
I_{yz}	-1.736000 x 10^{-8} m^4
k_y	1.995341 x 10^7 $1/m^4$
k_z	1.200106 x 10^8 $1/m^4$
k_{yz}	-2.797991 x 10^7 $1/m^4$
Neutral-axis angle	166.876195°

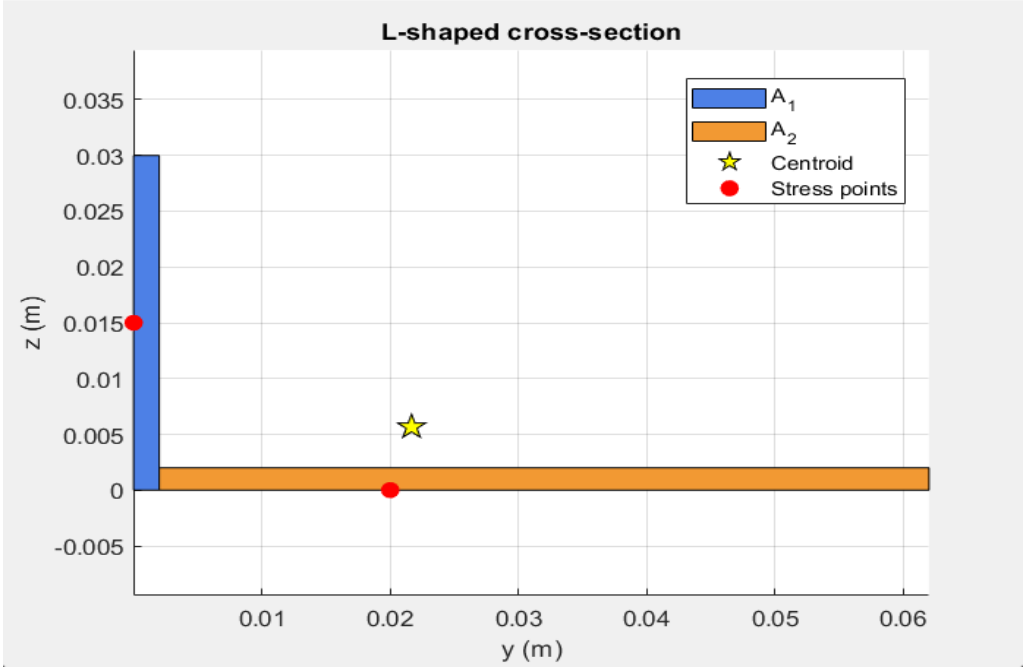


Fig. 3. L-Shaped Cross-Section with Centroid and Stress Points

The bending stress results at the ten required strain-gage locations are listed in Table 4. The bending stress was compressive at the locations with (y, z)=(0, 0.015); on the other hand, the bending stress was tensile at the locations with (y, z)=(0.020, 0). The largest bending stress magnitudes occurred close to the fixed end of the beam and decreased to a magnitude of zero as it approached the free end of the beam. This trend is consistent with the cantilever beam bending moment distribution, which is that the largest bending stress magnitude occurs at the root of the beam and becomes zero at the tip. The linear variation of bending moment along the beam is

shown in Fig. 4. The corresponding stress variation at the two selected cross-section locations is shown in Fig. 5.

Table 4. Calculated Bending Stress at the Ten Table 2 Strain-Gage Locations

Strain Gage	x (m)	y (m)	z (m)	Bending Stress (MPa)
1	0.0348	0.0000	0.0150	-26.475013
2	0.2761	0.0000	0.0150	-19.856260
3	0.5047	0.0000	0.0150	-13.585862
4	0.7333	0.0000	0.0150	-7.315464
5	0.9746	0.0000	0.0150	-0.696711
6	0.0348	0.0200	0.0000	37.440022
7	0.2761	0.0200	0.0000	28.080017
8	0.5047	0.0200	0.0000	19.212643
9	0.7333	0.0200	0.0000	10.345269
10	0.9746	0.0200	0.0000	0.985264

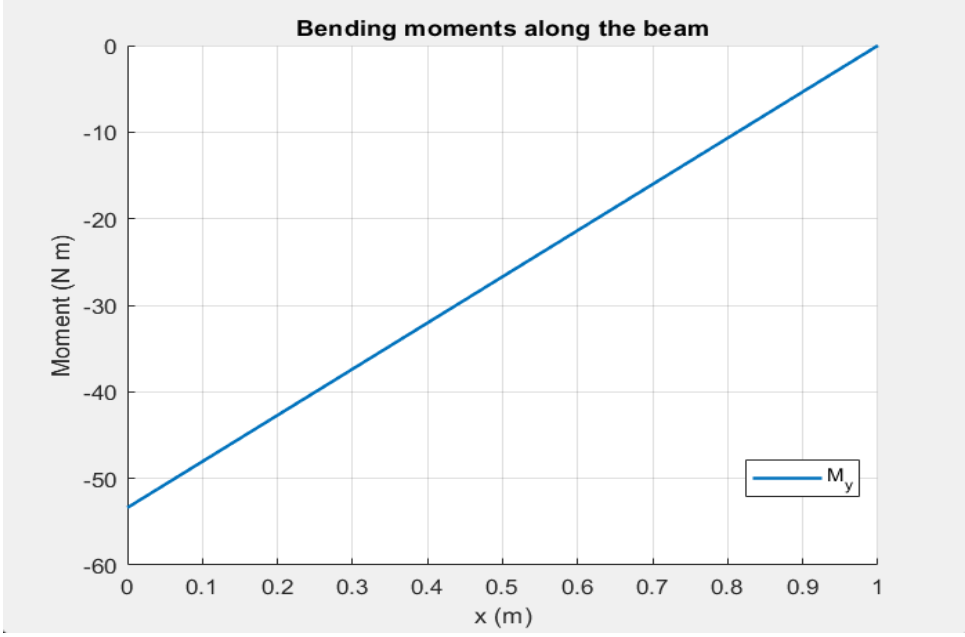


Fig. 4. Bending Moment Distribution along the Cantilever Beam

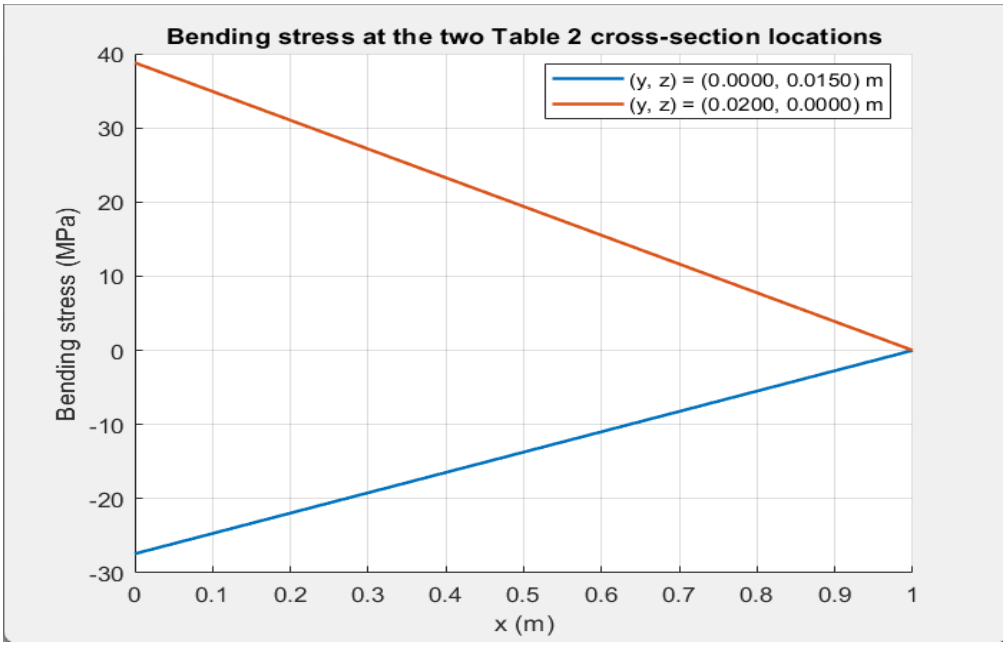


Fig. 5. Bending Stress Variation along the Beam at the Two Table 2 Cross-Section Locations

The horizontal and vertical displacement results at the five required cross-sections are listed in Table 5. The magnitude of the vertical displacement was found to be greater than the magnitude of the horizontal displacement at every reported location. At the farthest listed point from the fixed end of the beam, x=0.9746 m, the horizontal displacement was -6.950357 mm, and the vertical displacement was -29.811266 mm. The horizontal and vertical deflection distributions along the cantilever beam and the sampled strain-gage locations are shown in Fig. 6. The curve representing vertical deflection shows the dominant bending response. The nonzero curve representing horizontal deflection indicates coupling caused by the unsymmetrical L-shaped cross-section.

Table 5. Calculated Horizontal and Vertical Displacements at the Five Required Cross-Sections

Strain Gage	x (m)	Horizontal Displacement (mm)	Vertical Displacement (mm)
1	0.0348	-0.012973	-0.055645
2	0.2761	-0.750183	-3.217665
3	0.5047	-2.296322	-9.849318
4	0.7333	-4.403529	-18.887488
5	0.9746	-6.950357	-29.811266

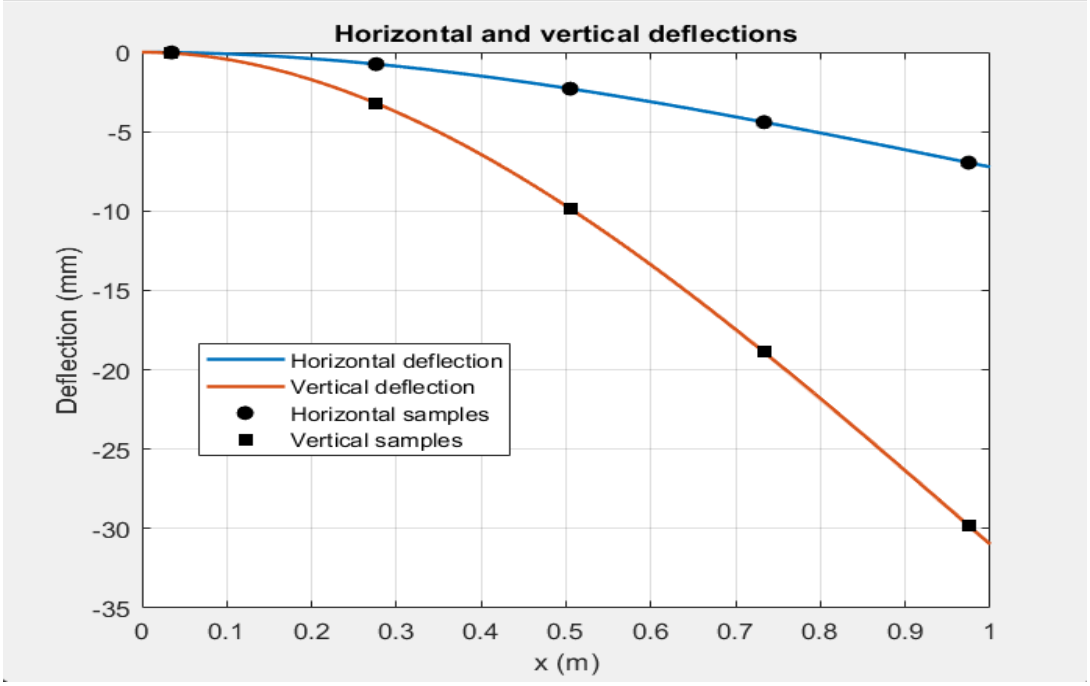


Fig. 6. Horizontal and Vertical Deflection Distributions along the Cantilever Beam

Overall, the results produced by the MATLAB program include the following requirements: section properties, bending stresses, and displacement values for the specified cantilever beam. The stress and displacement plots generated using the 1 mm discretization also show physically consistent behavior for a cantilever beam subjected to an applied tip load.

VI. Discussion

The results are consistent with the expected response of a cantilever beam with a tip load that was theoretically predicted. The bending moment is the largest at the fixed end of the beam and gradually decreases to zero as it approaches the free end of the beam, and so, the largest stress magnitudes occur close to the root of the beam and decrease towards the tip. One of the evaluated locations remained in compression; however, the other evaluated location remained in tension over the beam length. This agrees with the bending distribution shown in the results produced by the MATLAB program.

The results for displacement also show reasonable agreement with the theoretically predicted beam response. The vertical displacement was much larger than the horizontal displacement because the applied load at the tip of the beam acts in the vertical direction. Even so, the horizontal displacement was not zero because the L-shaped section is unsymmetrical and has a nonzero product of inertia. This causes coupling bending behavior. Overall, the results show that the MATLAB program produced physically consistent section property, stress, and deflection results for the required beam case.

VII. Conclusion

A MATLAB program for bidirectional bending analysis of an unsymmetrical L-shaped cantilever beam was successfully developed and applied to the specific beam in Task 2. The required cross-section properties, bending stresses, displacement results, and plots were

computed by the program. These results were found to be consistent with the expected cantilever beam behavior predicted by theory.

AEM341 Project Part1

Collin Atkins 4/24/2026

```
function results = AEM341_Project1_Main(inputs)
```

Inputs

```
    if nargin == 0
        inputs = build_default_inputs();
    end

    validate_inputs(inputs);

    beam = inputs.beam;

    loads = inputs.loads;

    analysis = inputs.analysis;

    section_positions = analysis.section_positions(:);

    sample_positions = analysis.sample_positions(:);
```

Cross-Section Properties

```
    section = get_l_section_properties(inputs.section);

    validate_stress_points(inputs.analysis.stress_points, section);
```

Bending Moments

```
    moment_y = get_bending_moment_y( ...
        beam.length, ...
        loads.tip_force_z, ...
        section_positions);

    moment_z = get_bending_moment_z( ...
        beam.length, ...
        loads.tip_force_y, ...
        section_positions);

     neutral_axis_angle = get_neutral_axis_angle( ...
        moment_y, ...
        moment_z, ...
        section);
```

Bending Stresses

```
    stress_values = get_stress_history( ...
        section, ...
```

```
    moment_y, ...
    moment_z, ...
    analysis.stress_points);

gauge_positions = analysis.sample_positions;

gauge_moment_y = get_bending_moment_y( ...
    beam.length, ...
    loads.tip_force_z, ...
    gauge_positions);

gauge_moment_z = get_bending_moment_z( ...
    beam.length, ...
    loads.tip_force_y, ...
    gauge_positions);

gauge_stress_values = get_stress_history( ...
    section, ...
    gauge_moment_y, ...
    gauge_moment_z, ...
    analysis.stress_points);
```

Deflections

```
[horizontal_disp, vertical_disp] = get_tip_load_deflections( ...
    section, ...
    beam, ...
    loads, ...
    section_positions);

[sample_horizontal, sample_vertical] = get_tip_load_deflections( ...
    section, ...
    beam, ...
    loads, ...
    sample_positions);
```

Outputs

```
stress_table = build_stress_table( ...
    gauge_positions, ...
    gauge_stress_values);

displacement_table = build_displacement_table( ...
    gauge_positions, ...
    sample_horizontal, ...
    sample_vertical);

results = struct();

results.inputs = inputs;
results.section = section;
results.section_positions = section_positions;
results.moment_y = moment_y;
```

```
    results.moment_z = moment_z;
    results.stress_points = analysis.stress_points;
    results.stress_values = stress_values;
    results.horizontal_disp = horizontal_disp;
    results.vertical_disp = vertical_disp;
    results.sample_positions = sample_positions;
    results.sample_horizontal = sample_horizontal;
    results.sample_vertical = sample_vertical;
    results.neutral_axis_angle_rad = neutral_axis_angle;
    results.neutral_axis_angle_deg = rad2deg(neutral_axis_angle);
    results.stress_table = stress_table;
    results.displacement_table = displacement_table;

    display_task_2_tables(stress_table, displacement_table);
    display_summary(results);
```

```
================ TASK 2 STRESS RESULTS ==================
Gage      x (m)      y (m)      z (m)     Stress (MPa)
   1      0.0348     0.0000     0.0150     -26.475013
   2      0.2761     0.0000     0.0150     -19.856260
   3      0.5047     0.0000     0.0150     -13.585862
   4      0.7333     0.0000     0.0150      -7.315464
   5      0.9746     0.0000     0.0150      -0.696711
   6      0.0348     0.0200     0.0000      37.440022
   7      0.2761     0.0200     0.0000      28.080017
   8      0.5047     0.0200     0.0000      19.212643
   9      0.7333     0.0200     0.0000      10.345269
  10      0.9746     0.0200     0.0000       0.985264

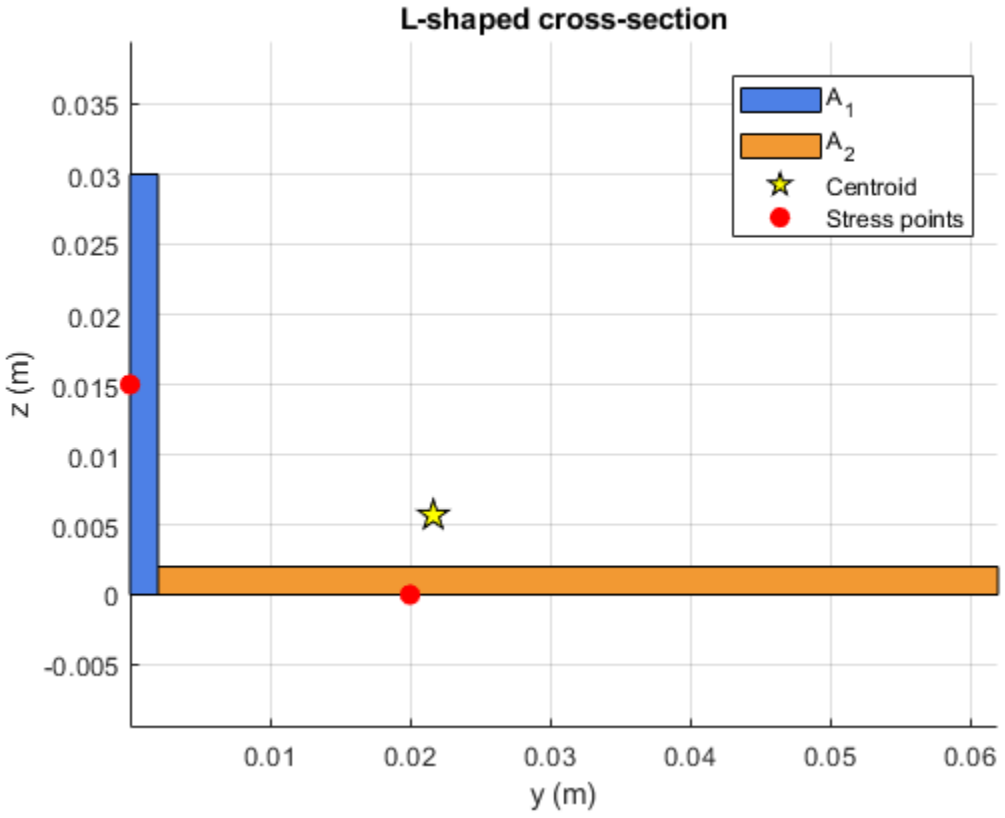
============== TASK 2 DISPLACEMENT RESULTS ==============
Gage      x (m)      y disp (mm)      z disp (mm)
   1      0.0348      -0.012973      -0.055645
   2      0.2761      -0.750183      -3.217665
   3      0.5047      -2.296322      -9.849318
   4      0.7333      -4.403529     -18.887488
   5      0.9746      -6.950357     -29.811266

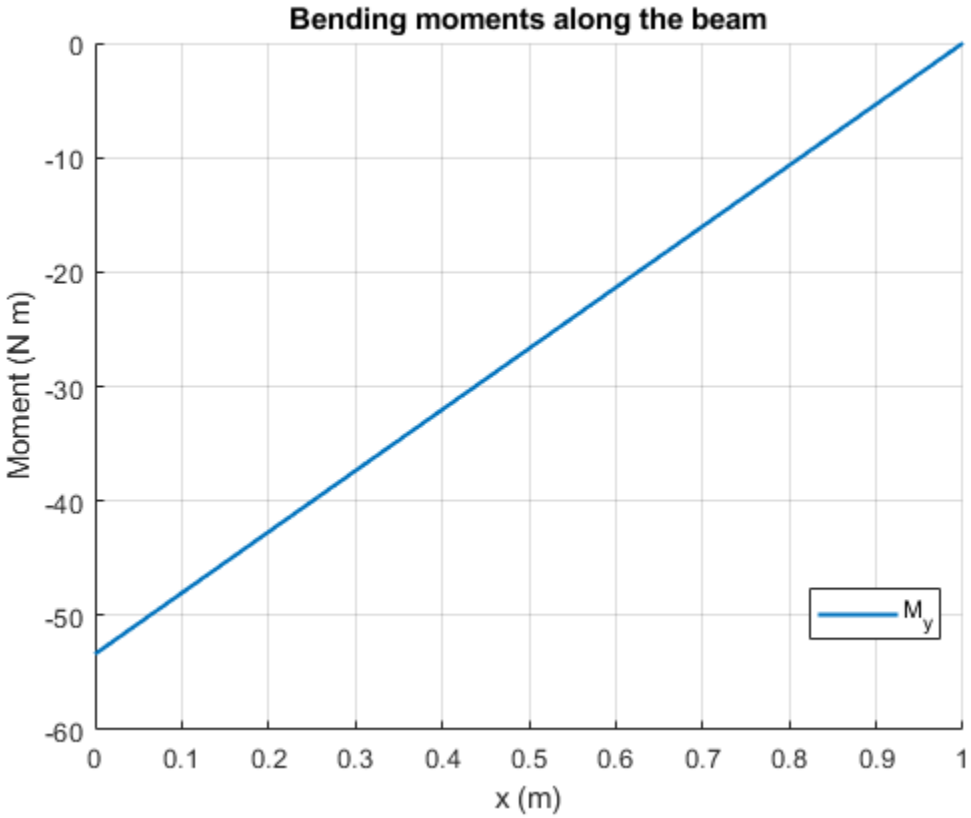
================ CROSS-SECTION PROPERTIES ===============
Area                     = 1.800000e-04 m^2
Centroid y-coordinate    = 2.166667e-02 m
Centroid z-coordinate    = 5.666667e-03 m
I_y                      = 1.238000e-08 m^4
I_z                      = 7.446000e-08 m^4
I_yz                     = -1.736000e-08 m^4
k_y                      = 1.995341e+07 1/m^4
k_z                      = 1.200106e+08 1/m^4
k_yz                     = -2.797991e+07 1/m^4
Neutral-axis angle       = 166.876195 deg

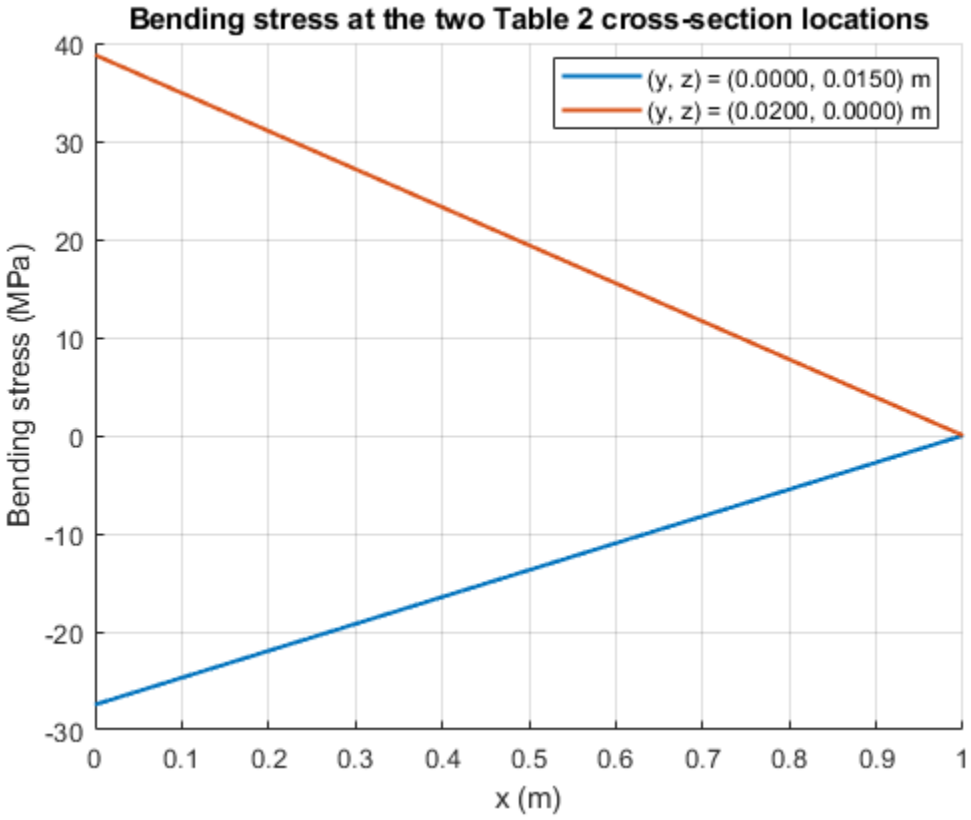
================ TIP DEFLECTIONS ========================
Horizontal tip defl.     = -7.225593e-03 m
Vertical tip defl.       = -3.099180e-02 m
```

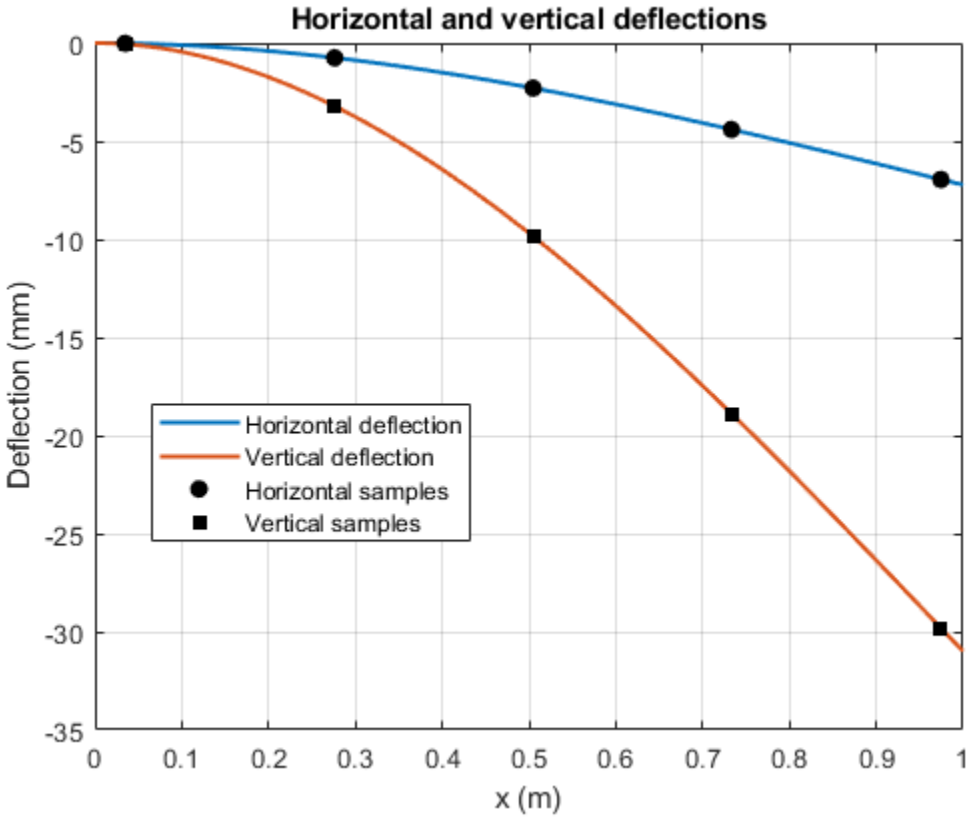
Plots

```
plot_cross_section(section, analysis.stress_points);
plot_bending_moments(section_positions, moment_y, moment_z);
plot_stress_history( ...
    section_positions, ...
    stress_values, ...
    analysis.stress_points);
plot_deflections( ...
    section_positions, ...
    horizontal_disp, ...
    vertical_disp, ...
    sample_positions, ...
    sample_horizontal, ...
    sample_vertical);
```




Bending moments along the beam
Moment (N m)
x (m)
M_y
0
-10
-20
-30
-40
-50
-60
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1


Bending stress at the two Table 2 cross-section locations
Bending stress (MPa)
x (m)
(y, z) = (0.0000, 0.0150) m
(y, z) = (0.0200, 0.0000) m
40
30
20
10
0
-10
-20
-30
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1




```
end
```

Local Functions

```
function inputs = build_default_inputs()

    POUND_TO_NEWTON = 4.4482216152605;

    inputs = struct();

    inputs.section = struct();

    inputs.section.h1 = 0.0300;
    inputs.section.h2 = 0.0600;
    inputs.section.t1 = 0.0020;
    inputs.section.t2 = 0.0020;

    inputs.beam = struct();

    inputs.beam.length = 1.0000;
    inputs.beam.elastic_modulus = 68.9e9;

    inputs.loads = struct();

    inputs.loads.tip_force_y = 0.0;
    inputs.loads.tip_force_z = -12.0 * POUND_TO_NEWTON;
```

```
    inputs.analysis = struct();

    inputs.analysis.section_positions = (0.0000:0.0010:1.0000).';

    inputs.analysis.sample_positions = [ ...
        0.0348; ...
        0.2761; ...
        0.5047; ...
        0.7333; ...
        0.9746];

    inputs.analysis.stress_points = [ ...
        0.0000, 0.0150; ...
        0.0200, 0.0000];
end


function validate_inputs(inputs)

    if ~isfield(inputs, 'section') ...
            || ~isfield(inputs, 'beam') ...
            || ~isfield(inputs, 'loads') ...
            || ~isfield(inputs, 'analysis')
        error('The inputs structure is missing required fields.');
    end

    required_section = {'h1', 'h2', 't1', 't2'};

    for field_ind = 1:numel(required_section)
        current_field = required_section{field_ind};

        if ~isfield(inputs.section, current_field)
            error('The section field %s is missing.', current_field);
        end
    end

    if inputs.section.h1 <= 0.0 || inputs.section.h2 <= 0.0
        error('The section heights must be positive.');
    end

    if inputs.section.t1 <= 0.0 || inputs.section.t2 <= 0.0
        error('The section thicknesses must be positive.');
    end

    if inputs.beam.length <= 0.0
        error('The beam length must be positive.');
    end

    if inputs.beam.elastic_modulus <= 0.0
        error('The elastic modulus must be positive.');
    end

    if any(inputs.analysis.section_positions < 0.0) ...
```

```
            || any(inputs.analysis.section_positions > inputs.beam.length)
        error('All section positions must lie between 0 and beam length.');
    end

    if any(inputs.analysis.sample_positions < 0.0) ...
            || any(inputs.analysis.sample_positions > inputs.beam.length)
        error('All sample positions must lie between 0 and beam length.');
    end

    if size(inputs.analysis.stress_points, 2) ~= 2
        error('The stress_points array must have two columns.');
    end
end


function section = get_l_section_properties(section)

    area_1 = section.t1 * section.h1;

    area_2 = section.h2 * section.t2;

    total_area = area_1 + area_2;

    y_1 = section.t1 / 2.0;

    z_1 = section.h1 / 2.0;

    y_2 = section.t1 + section.h2 / 2.0;

    z_2 = section.t2 / 2.0;

    y_centroid = (area_1 * y_1 + area_2 * y_2) / total_area;

    z_centroid = (area_1 * z_1 + area_2 * z_2) / total_area;

    inertia_y_1_local = section.t1 * section.h1^3 / 12.0;

    inertia_z_1_local = section.h1 * section.t1^3 / 12.0;

    inertia_y_2_local = section.h2 * section.t2^3 / 12.0;

    inertia_z_2_local = section.t2 * section.h2^3 / 12.0;

    delta_y_1 = y_1 - y_centroid;

    delta_z_1 = z_1 - z_centroid;

    delta_y_2 = y_2 - y_centroid;

    delta_z_2 = z_2 - z_centroid;

    inertia_y = inertia_y_1_local ...
        + area_1 * delta_z_1^2 ...
        + inertia_y_2_local ...
```

```
        + area_2 * delta_z_2^2;

    inertia_z = inertia_z_1_local ...
        + area_1 * delta_y_1^2 ...
        + inertia_z_2_local ...
        + area_2 * delta_y_2^2;

    inertia_yz = area_1 * delta_y_1 * delta_z_1 ...
        + area_2 * delta_y_2 * delta_z_2;

    denominator = inertia_y * inertia_z - inertia_yz^2;

    if denominator <= 0.0
        error('The inertia denominator must be positive.');
    end

    section.area_1 = area_1;

    section.area_2 = area_2;

    section.area = total_area;

    section.y_1 = y_1;

    section.z_1 = z_1;

    section.y_2 = y_2;

    section.z_2 = z_2;

    section.y_centroid = y_centroid;

    section.z_centroid = z_centroid;

    section.inertia_y = inertia_y;

    section.inertia_z = inertia_z;

    section.inertia_yz = inertia_yz;

    section.denominator = denominator;

    section.k_y = inertia_y / denominator;

    section.k_z = inertia_z / denominator;

    section.k_yz = inertia_yz / denominator;
end


function validate_stress_points(stress_points, section)

    num_points = size(stress_points, 1);
```

```
    for point_ind = 1:num_points
        point_y = stress_points(point_ind, 1);

        point_z = stress_points(point_ind, 2);

        if ~is_in_l_section(point_y, point_z, section)
            error( ...
                'Stress point (%g, %g) is outside the L-section.', ...
                point_y, ...
                point_z);
        end
    end
end


function is_inside = is_in_l_section(point_y, point_z, section)

    inside_vertical = point_y >= 0.0 ...
        && point_y <= section.t1 ...
        && point_z >= 0.0 ...
        && point_z <= section.h1;

    inside_horizontal = point_y >= section.t1 ...
        && point_y <= section.t1 + section.h2 ...
        && point_z >= 0.0 ...
        && point_z <= section.t2;

    is_inside = inside_vertical || inside_horizontal;
end


function moment_y = get_bending_moment_y( ...
    beam_length, ...
    tip_force_z, ...
    section_positions)

    moment_y = tip_force_z .* (beam_length - section_positions);
end


function moment_z = get_bending_moment_z( ...
    beam_length, ...
    tip_force_y, ...
    section_positions)

    moment_z = tip_force_y .* (beam_length - section_positions);
end


function stress_values = get_stress_history( ...
    section, ...
    moment_y, ...
    moment_z, ...
    stress_points)
```

```
    num_sections = numel(moment_y);

    num_points = size(stress_points, 1);

    stress_values = zeros(num_sections, num_points);

    for point_ind = 1:num_points
        point_y = stress_points(point_ind, 1);

        point_z = stress_points(point_ind, 2);

        y_rel = point_y - section.y_centroid;

        z_rel = point_z - section.z_centroid;

        term_y = moment_z .* section.inertia_y ...
            - moment_y .* section.inertia_yz;

        term_z = moment_y .* section.inertia_z ...
            - moment_z .* section.inertia_yz;

        stress_values(:, point_ind) = ( ...
            term_y .* y_rel ...
            + term_z .* z_rel) ./ section.denominator;
    end
end


function [horizontal_disp, vertical_disp] = get_tip_load_deflections( ...
    section, ...
    beam, ...
    loads, ...
    positions)

    shape_function = positions.^2 .* (3.0 * beam.length - positions) ...
        / 6.0;

    horizontal_coef = ( ...
        section.inertia_y * loads.tip_force_y ...
        - section.inertia_yz * loads.tip_force_z) ...
        / (beam.elastic_modulus * section.denominator);

    vertical_coef = ( ...
        section.inertia_z * loads.tip_force_z ...
        - section.inertia_yz * loads.tip_force_y) ...
        / (beam.elastic_modulus * section.denominator);

    horizontal_disp = horizontal_coef .* shape_function;

    vertical_disp = vertical_coef .* shape_function;
end

function neutral_axis_angle = get_neutral_axis_angle( ...
```

```
    moment_y, ...
    moment_z, ...
    section)

    neutral_axis_angle = nan(size(moment_y));

    active_mask = abs(moment_y) > 0.0 | abs(moment_z) > 0.0;

    numerator = -(moment_z(active_mask) * section.inertia_y ...
        - moment_y(active_mask) .* section.inertia_yz);

    denominator = moment_y(active_mask) * section.inertia_z ...
        - moment_z(active_mask) .* section.inertia_yz;

    neutral_axis_angle(active_mask) = atan2(numerator, denominator);
end

function display_summary(results)

    section = results.section;

    fprintf('\n');
    fprintf('================ CROSS-SECTION PROPERTIES ==============\n');
    fprintf('Area                      = %.6e m^2\n', section.area);
    fprintf('Centroid y-coordinate     = %.6e m\n', section.y_centroid);
    fprintf('Centroid z-coordinate     = %.6e m\n', section.z_centroid);
    fprintf('I_y                       = %.6e m^4\n', section.inertia_y);
    fprintf('I_z                       = %.6e m^4\n', section.inertia_z);
    fprintf('I_yz                      = %.6e m^4\n', section.inertia_yz);
    fprintf('k_y                       = %.6e 1/m^4\n', section.k_y);
    fprintf('k_z                       = %.6e 1/m^4\n', section.k_z);
    fprintf('k_yz                      = %.6e 1/m^4\n', section.k_yz);

    first_angle_ind = find(~isnan(results.neutral_axis_angle_deg), 1);

    if ~isempty(first_angle_ind)
        fprintf('Neutral-axis angle      = %.6f deg\n', ...
            results.neutral_axis_angle_deg(first_angle_ind));
    else
        fprintf('Neutral-axis angle      = NaN deg\n');
    end

    fprintf('\n');
    fprintf('================ TIP DEFLECTIONS ======================\n');
    fprintf('Horizontal tip defl.    = %.6e m\n', ...
        results.horizontal_disp(end));
    fprintf('Vertical tip defl.      = %.6e m\n', ...
        results.vertical_disp(end));

end


function plot_cross_section(section, stress_points)
```

```
    figure('Name', 'L-Section Geometry');

    hold on;

    handle_a1 = patch( ...
        [0.0, section.t1, section.t1, 0.0], ...
        [0.0, 0.0, section.h1, section.h1], ...
        [0.3, 0.5, 0.9], ...
        'EdgeColor', 'k', ...
        'DisplayName', 'A_1');

    handle_a2 = patch( ...
        [section.t1, section.t1 + section.h2, ...
        section.t1 + section.h2, section.t1], ...
        [0.0, 0.0, section.t2, section.t2], ...
        [0.95, 0.6, 0.2], ...
        'EdgeColor', 'k', ...
        'DisplayName', 'A_2');

    handle_centroid = plot( ...
        section.y_centroid, ...
        section.z_centroid, ...
        'kp', ...
        'MarkerSize', 12, ...
        'MarkerFaceColor', 'y', ...
        'DisplayName', 'Centroid');

    handle_stress = plot( ...
        stress_points(:, 1), ...
        stress_points(:, 2), ...
        'ro', ...
        'MarkerSize', 7, ...
        'MarkerFaceColor', 'r', ...
        'DisplayName', 'Stress points');

    axis equal;

    grid on;

    xlabel('y (m)');

    ylabel('z (m)');

    title('L-shaped cross-section');

    legend( ...
        [handle_a1; handle_a2; handle_centroid; handle_stress], ...
        'Location', ...
        'best');

    hold off;
end
```

```
function plot_bending_moments(section_positions, moment_y, moment_z)

    figure('Name', 'Bending Moments');

    hold on;

    plot(section_positions, moment_y, 'LineWidth', 1.5, ...
        'DisplayName', 'M_y');

    if max(abs(moment_z)) > 1.0e-12
        plot(section_positions, moment_z, '--', 'LineWidth', 1.5, ...
            'DisplayName', 'M_z');
    end

    grid on;

    xlabel('x (m)');

    ylabel('Moment (N m)');

    title('Bending moments along the beam');

    legend('Location', 'best');

    hold off;
end


function plot_stress_history( ...
    section_positions, ...
    stress_values, ...
    stress_points)

    figure('Name', 'Stress History');

    hold on;

    num_points = size(stress_points, 1);

    point_labels = build_stress_labels(stress_points);

    for point_ind = 1:num_points
        plot( ...
            section_positions, ...
            stress_values(:, point_ind) / 1.0e6, ...
            'LineWidth', ...
            1.5, ...
            'DisplayName', ...
            point_labels{point_ind});
    end

    grid on;

    xlabel('x (m)');
```

```
    ylabel('Bending stress (MPa)');

    title('Bending stress at the two Table 2 cross-section locations');

    legend('Location', 'best');

    hold off;
end


function plot_deflections( ...
    section_positions, ...
    horizontal_disp, ...
    vertical_disp, ...
    sample_positions, ...
    sample_horizontal, ...
    sample_vertical)

    figure('Name', 'Deflections');

    plot(section_positions, horizontal_disp * 1.0e3, 'LineWidth', 1.5);

    hold on;

    plot(section_positions, vertical_disp * 1.0e3, 'LineWidth', 1.5);

    plot( ...
        sample_positions, ...
        sample_horizontal * 1.0e3, ...
        'ko', ...
        'MarkerFaceColor', ...
        'k');

    plot( ...
        sample_positions, ...
        sample_vertical * 1.0e3, ...
        'ks', ...
        'MarkerFaceColor', ...
        'k');

    grid on;

    xlabel('x (m)');

    ylabel('Deflection (mm)');

    title('Horizontal and vertical deflections');

    legend( ...
        'Horizontal deflection', ...
        'Vertical deflection', ...
        'Horizontal samples', ...
        'Vertical samples', ...
```

```
        'Location', ...
        'best');

    hold off;
end


function point_labels = build_stress_labels(stress_points)

    num_points = size(stress_points, 1);

    point_labels = cell(num_points, 1);

    for point_ind = 1:num_points
        point_labels{point_ind} = sprintf( ...
            '(y, z) = (%.4f, %.4f) m', ...
            stress_points(point_ind, 1), ...
            stress_points(point_ind, 2));
    end
end

function stress_table = build_stress_table( ...
    gauge_positions, ...
    gauge_stress_values)

    num_positions = numel(gauge_positions);

    strain_gage = (1:10).';

    x_value = [gauge_positions; gauge_positions];

    y_value = [ ...
        zeros(num_positions, 1); ...
        0.0200 * ones(num_positions, 1)];

    z_value = [ ...
        0.0150 * ones(num_positions, 1); ...
        zeros(num_positions, 1)];

    stress_pa = [ ...
        gauge_stress_values(:, 1); ...
        gauge_stress_values(:, 2)];

    stress_mpa = stress_pa / 1.0e6;

    stress_table = table( ...
        strain_gage, ...
        x_value, ...
        y_value, ...
        z_value, ...
        stress_pa, ...
        stress_mpa);
end
```

```
function displacement_table = build_displacement_table( ...
    gauge_positions, ...
    sample_horizontal, ...
    sample_vertical)

    strain_gage = (1:5).';

    x_value = gauge_positions;

    horizontal_disp_m = sample_horizontal;

    vertical_disp_m = sample_vertical;

    horizontal_disp_mm = sample_horizontal * 1.0e3;

    vertical_disp_mm = sample_vertical * 1.0e3;

    displacement_table = table( ...
        strain_gage, ...
        x_value, ...
        horizontal_disp_m, ...
        vertical_disp_m, ...
        horizontal_disp_mm, ...
        vertical_disp_mm);
end

function display_task_2_tables(stress_table, displacement_table)

    fprintf('\n');
    fprintf('================ TASK 2 STRESS RESULTS ==================\n');
    fprintf('Gage      x (m)      y (m)      z (m)     Stress (MPa)\n');

    for row_ind = 1:height(stress_table)
        fprintf( ...
            '%4d   %9.4f   %8.4f   %8.4f   %12.6f\n', ...
            stress_table.strain_gage(row_ind), ...
            stress_table.x_value(row_ind), ...
            stress_table.y_value(row_ind), ...
            stress_table.z_value(row_ind), ...
            stress_table.stress_mpa(row_ind));
    end

    fprintf('\n');
    fprintf('============== TASK 2 DISPLACEMENT RESULTS ==============\n');
    fprintf('Gage      x (m)      y disp (mm)      z disp (mm)\n');

    for row_ind = 1:height(displacement_table)
        fprintf( ...
            '%4d   %9.4f   %12.6f   %12.6f\n', ...
            displacement_table.strain_gage(row_ind), ...
            displacement_table.x_value(row_ind), ...
            displacement_table.horizontal_disp_mm(row_ind), ...
            displacement_table.vertical_disp_mm(row_ind));
```

```
    end
end
```

```
ans =

  struct with fields:

                    inputs: [1×1 struct]
                   section: [1×1 struct]
         section_positions: [1001×1 double]
                  moment_y: [1001×1 double]
                  moment_z: [1001×1 double]
             stress_points: [2×2 double]
             stress_values: [1001×2 double]
           horizontal_disp: [1001×1 double]
             vertical_disp: [1001×1 double]
          sample_positions: [5×1 double]
         sample_horizontal: [5×1 double]
           sample_vertical: [5×1 double]
    neutral_axis_angle_rad: [1001×1 double]
    neutral_axis_angle_deg: [1001×1 double]
              stress_table: [10×6 table]
        displacement_table: [5×6 table]
```

AI Use Statement

For this assignment's preparation, the author utilized ChatGPT, a language model created by OpenAI. Within this assignment, ChatGPT was used for brainstorming, checking MATLAB code organization, identifying possible errors, polishing code, and suggesting debugging fixes.

Published with MATLAB® R2023b